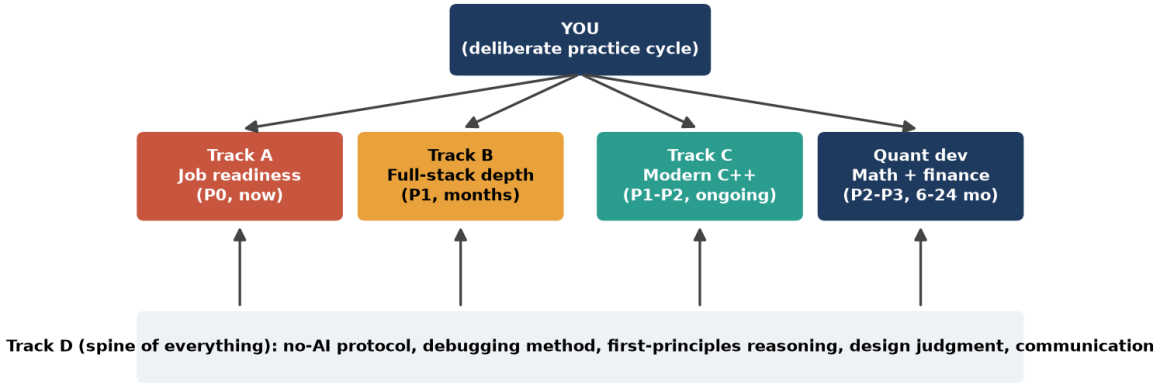
The Elite Engineer

Master Blueprint, Diagnostic, Survival Plan and Lesson 1

From working developer to full-stack engineer, modern C++ specialist and quantitative developer

Figure 1. Four tracks, one training system



Volume 0 of the training system. Prepared for Frank. Start date: next Monday.

1. Introduction: how this book works

This is not a textbook to read. It is a training system. The measure of progress is what you can **do without help**, and every lesson produces a deliverable you must build, test and defend.

The learning cycle

```
Understand -> Recall -> Predict -> Implement -> Test -> Debug -> Explain -> Review -> Rebuild
```

Reading counts as *Introduced* at most. Copying code counts as nothing. A topic is mastered when you can reproduce, adapt, explain and defend a solution after a delay and without the original explanation.

The three work modes

Mode	Allowed	Trains
A: Closed-book	Your head, editor, compiler/interpreter, your own notes. No AI, no tutorials.	Recall, fluency, problem solving
B: Docs-only	Official docs, debugger, test and compiler output. No AI design or code.	Learning unfamiliar tech independently
C: AI-assisted	AI as reviewer, explainer, brainstorm partner. You own design, correctness, security, tests.	Leverage with judgment

Every exercise is tagged A, B or C. Default to A or B for anything labelled *Practice*. Use C at work only within your employer's policy, and never paste proprietary code or secrets into an external tool without permission.

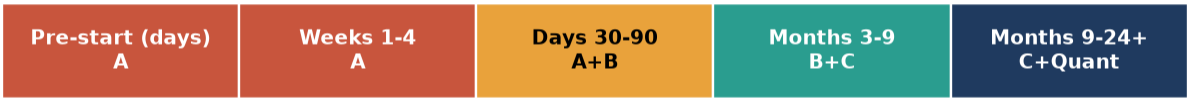
Priority labels

Label	Meaning	Example
P0	Needed now or an essential foundation	Git workflow, tracing a request, SQL basics, PHP/Python semantics
P1	Core for independent professional competence	Testing, auth, transactions, Moodle APIs, C++ ownership
P2	Advanced specialization	Concurrency, memory model, Monte Carlo, performance engineering
P3	Optional, direction-specific	Template metaprogramming, stochastic calculus, ABI details

Ground rules

- Understand an inherited system before changing it. Follow its conventions; do not import your preferences.
- Prefer the simplest design that meets the requirements. Unnecessary complexity is a defect.
- Measure before optimizing. Evidence before assertions.
- Time spent is not progress. Verified learning and shipped work are.
- Hard ceiling: sleep and your job come first. A sustainable 8-12 hours a week beats a 3-week burnout.

Figure 2. Three horizons: urgent first, ambitious always



Weekly time share (suggested, ~10 h/week study on top of work):

Job: 100% of study time	Job 70 / Foundations 30	Job 50 / Full-stack 30 / C++ 20	Full-stack 35 / C++ 40 / Math 25	C++ 35 / Quant 40 / Capstones 25

2. Diagnostic assessment

Do this in Mode A, with a timer: **90 minutes**, no search, no AI. Write answers on paper or in a file. Wrong answers and blanks are the point: they set your starting path. Score yourself after, then send me the answers and I will grade them and adjust the curriculum.

Python and FastAPI (P0)

- **P1.** Predict the output: `def f(x, acc=[]): acc.append(x); return acc`, then `f(1); f(2); print(f(3))`. Why?
- **P2.** What does `a = [[0]*2]*2; a[0][0] = 1` produce? Explain with references.
- **P3.** Write a decorator that logs the duration of a function call. Preserve the function's name and docstring.
- **P4.** In FastAPI, what is the difference between `def` and `async def` path operations? What breaks if you call `requests.get` inside an `async def`?
- **P5.** Explain `Depends()`. How would you replace the DB session in a test?
- **Practical (30 min):** From a blank directory, create a FastAPI app with `POST /items` and `GET /items/{id}`, an in-memory store, a Pydantic model that rejects negative prices, a 404 for unknown ids, and three pytest tests.

Databases and backend (P0/P1)

- **D1.** Given `users(id, name)` and `orders(id, user_id, total)`, write a query: each user's name and total spend, including users with no orders.
- **D2.** An index on `(a, b)`: does `WHERE b = 5` use it? Why?
- **D3.** Two requests read balance=100 and each subtract 30. Name two ways to prevent the lost update.
- **D4.** What is an N+1 query? How do you detect it?
- **D5.** Why does OFFSET pagination misbehave on a table under concurrent inserts, and what is the alternative?
- **D6.** Difference between authentication and authorization. Give one real broken-access-control bug.

PHP and Moodle (P0)

- **H1.** `0 == 'a'`, `'1' == '01'`, `null == false`: results in PHP 8? When do you use `===`?
- **H2.** What do `declare(strict_types=1)` and typed properties change?
- **H3.** Moodle: what is a context, a capability and a role? Where is a capability defined?
- **H4.** Why use `$DB->get_records_sql` with placeholders rather than string interpolation?
- **H5.** Name the files required for a minimal Moodle local plugin and what each does.
- **H6.** What do `require_login()`, `require_capability()`, `required_param()`/`optional_param()` and `sesskey` each protect against?

Engineering practice (P0)

- **E1.** You are handed an unfamiliar repo and a bug ticket. Write your first 10 actions in order.
- **E2.** Write the tests (not the code) for `slugify(title)`. Which 8 cases and why?
- **E3.** Explain `git rebase` vs `merge`, and when `bisect` and `reflog` save you.
- **E4.** A test passes locally and fails in CI. List five hypotheses and how you would test each.

C++ and algorithms (P1)

- **C1.** What is RAII? What are three bugs in `int* p = new int[10]; /*...*/ delete p;`?
- **C2.** What does `std::move` actually do? What state is a moved-from `std::string` in?
- **C3.** Why is iterating a `std::vector<int>` usually much faster than `std::list<int>`?
- **C4.** Compile and link: what are a translation unit, an ODR violation and a header guard?

- **A1.** Implement binary search from memory. State the loop invariant and test at least 5 boundary cases.
- **A2.** Explain amortized O(1) `push_back` and why doubling matters.
- **A3.** Two-sum: give an $O(n^2)$ and an O(n) solution and their trade-offs.

Mathematics and quant finance (P2)

- **M1.** A test is 99% sensitive and 95% specific; prevalence is 1%. Compute P(disease | positive).
- **M2.** Expectation and variance of a fair six-sided die. Of the sum of two dice.
- **M3.** Multiply [[1,2],[3,4]] by [[0,1],[1,0]]. What does the result do to the columns?
- **M4.** Derivative of `x*ln(x)`; minimum of x^2 `- 4x + 7`.
- **M5.** Why is `0.1 + 0.2 == 0.3` false in floating point? How should you compare?
- **Q1.** Simple vs log returns: why are log returns additive? Compute both for 100 -> 110 -> 99.
- **Q2.** What is the payoff of a long European call and a long put at expiry? Sketch them.
- **Q3.** Name three biases that make a backtest look better than reality.

Self-report (1-5)

Rate yourself: Linux CLI, Docker, HTTP/REST, SQL joins, writing tests, reading other people's code, Big-O, and "percentage of my recent code I could rebuild without AI." The last number is the most important baseline.

Scoring and routing

Score in section	Interpretation	Route
80%+ and practical works	Solid	Skim lesson, do mastery gate only
50-79%	Familiar, gaps in internals	Full lesson, failure lab emphasised
< 50%	Foundation gap	Bridge lesson first, then main lesson

3. Prioritized curriculum

Lessons are numbered by volume. P-labels say when to take them; the reason is the justification. Each lesson follows the 12-step training-session format and ends with a deliverable.

#	Module	Pri	Why this priority	Deliverable
0.1	Day-one protocol: setup, run, test, trace a request	P0	You start Monday; this determines week 1	Repo map + request trace
0.2	Git workflow: branches, diffs, rebase, PRs, bisect	P0	Every change you ship goes through it	Clean PR from a practice repo
0.3	Debugging method and failure log	P0	Most of the job is reading failures	Debug report on a broken app
0.4	Communicating progress, blockers, asking questions	P0	Cheapest high-leverage skill	Standup + blocker message templates
1.1	Python object model, mutability, scope, closures	P0	Source of most subtle bugs	Bug-hunt: 10 broken snippets
1.2	Iterators, generators, context managers, exceptions	P0	Idiomatic backend code uses all	Streaming CSV processor
1.3	Types, packaging, venv, pyproject, config, logging	P0	Reproducible environments	Package with CI-ready layout
1.4	pytest, fixtures, mocking, isolation	P0	Safe changes need tests	Test suite for legacy function
2.1	HTTP, REST, JSON, status codes, API contracts	P0	Shared language of backend	OpenAPI-first design
2.2	FastAPI lifecycle, routing, Pydantic, DI	P0	Your Python framework	CRUD API from blank repo
2.3	SQL, schema design, joins, indexes, EXPLAIN	P0	Every service has a DB	Schema + query-plan report
2.4	Transactions, isolation, migrations, pagination	P1	Correctness under concurrency	Race-condition lab
2.5	AuthN/Z, passwords, JWT/OAuth basics, secrets	P1	Security baseline	Auth + threat model
2.6	Async, blocking calls, background jobs, retries, idempotency	P1	Failure behaviour	Idempotent job endpoint
2.7	Observability, deploy, health checks, timeouts	P1	Operational maturity	Capstone A
3.1	Modern PHP semantics, types, OOP, namespaces, Composer	P0	Moodle is PHP	Typed PHP mini-library
3.2	Moodle architecture, plugin types, directory layout	P0	Locate code and extension points	Plugin skeleton
3.3	Capabilities, contexts, roles, access control	P0	Most Moodle security bugs live here	Permissioned page
3.4	Forms API, params, output API, strings, DB API	P1	Core plugin work	CRUD plugin
3.5	Events, tasks, web services, caching, files, privacy API	P1	Production plugins need these	Service + task
3.6	PHPUnit, Behat, coding style, upgrades	P1	Quality and compatibility	Capstone B
4.1	Architecture: cohesion, boundaries, ports and adapters	P1	Judgment for changes	ADR + refactor proposal
4.2	Testing strategy, code review, refactoring legacy	P1	Leverage in inherited code	Review of a real PR
4.3	Security and reliability deep dive	P1	Responsible engineer	Security review report
4.4	Performance engineering method	P2	Measure before optimizing	Benchmark report

#	Module	Pri	Why this priority	Deliverable
5.1	C++ build, types, lifetime, references, const	P1	Foundation for track C	CLI tool
5.2	RAII, copy/move, smart pointers, ownership	P1	The core of modern C++	Resource-owning class
5.3	STL, iterators, lambdas, templates, concepts	P1	Daily C++ vocabulary	Generic container
5.4	DSA with implementations and invariants	P1	Interviews and real systems	Vector, hash map, heap
5.5	Sanitizers, UB, debugging, profiling	P1	Finds bugs you cannot see	Bug autopsy
5.6	Cache locality, allocators, concurrency, memory model	P2	Performance work	Threaded processor
6.1	Probability, distributions, expectation, Bayes	P2	Quant core	Probability simulator
6.2	Linear algebra, calculus, numerical methods, floating point	P2	Quant core	Numerics library
6.3	Statistics, regression, time series, Monte Carlo	P2	Model validation	MC experiment
6.4	Markets, returns, options, Black-Scholes, risk	P2	Domain knowledge	Pricer + risk analyzer
6.5	Backtesting, bias, costs, order book, market data	P2	Applied quant dev	Capstones C, D
7.1	AI-independence program and audits	P1	Runs across every volume	Audit scorecards
7.2	Design docs, ADRs, post-mortems, technical defense	P1	Judgment and communication	Written set

Deliberately deferred: template metaprogramming, lock-free data structures, stochastic calculus, ABI stability (P3). They will not appear until the P1 material is demonstrated.

4. First-week survival plan

You have only a few days. The goal is **not** to learn the stack; it is to arrive able to set up, read, ask good questions and make one small safe change. Budget 1.5-2.5 hours per evening. Anything unsaid about your employer's stack is an **assumption**: verify in the actual repo and internal docs.

When	Do (Mode)	Evidence it is done
Evening 1	Install Python 3.12+, venv, pip/uv, Git, Docker; run *one* public FastAPI example. Take the diagnostic (A).	Example runs; diagnostic answers written
Evening 2	Read a mid-size FastAPI repo; trace one request on paper (Figure 3). Write 5 questions you could not answer (B).	One-page trace diagram
Evening 3	Local Moodle via docker. Read a small core plugin (block or local): version.php, db/access.php, lib.php, classes/ (B).	Plugin file map with purpose of each file
Evening 4	SQL practice: 10 joins/aggregates on a sample DB; read one EXPLAIN. Git: branch, rebase, resolve a conflict (A).	Queries + a clean PR in a scratch repo
Evening 5	Write the day-one question list (below). Lesson 1 failure lab.	Question list + bug-hunt results
Weekend	Rest. Review failure log. Light recall quiz.	Sleep

Day-one checklist (print it)

- ■ Find the README, CONTRIBUTING, docs/, Makefile/justfile/scripts and CI config. Read them in that order.
- ■ Set up exactly as documented. Record every step that failed and what fixed it; fix the docs later if allowed.
- ■ Run the app. Run the tests. Note how long both take and which tests fail on a clean checkout.
- ■ Ask: how do I get env vars/secrets? which DB and version? what is the branch/PR/review process?
- ■ Locate: entry point, routing, config, data layer, tests. Trace one request all the way to the DB.
- ■ Pick a tiny, low-risk ticket. Reproduce first. Write a failing test. Change as little as possible. Verify.
- ■ Message at checkpoints: what you did, what you learned, what you will do next, where you are blocked.

Questions to ask in week one

- What does done mean here (tests, review, docs, deploy)? Who reviews my work?
- Which parts of the code are fragile or off-limits? Is there a style guide and linter config?
- How do releases and rollbacks happen? Where do logs and errors go? Who is on call?
- What is the policy on AI tools and on handling customer or student data?

5. The 30-90 day professional plan

Window	Goal	Learning focus	Proof
Days 1-5	Set up, read, ask	Lessons 0.1-0.4; Moodle skeleton; request tracing	First small PR merged
Days 6-30	Reliable small changes	1.1-1.4, 2.1-2.3, 3.1-3.3; write tests for every change; keep a failure log	3+ merged tickets with tests; one bug diagnosed with evidence
Days 31-60	Own a feature	2.4-2.6, 3.4-3.5, 4.1-4.2; review others' PRs	A feature end to end incl. migration, tests, docs
Days 61-90	Own quality and operations	2.7, 3.6, 4.3, 4.4; first design doc; first AI-dependence audit	Design doc, performance or security finding, audit passed

Rules: use work tasks as practice material where allowed; never copy employer code into personal repos. Do not refactor what you do not yet understand. Propose improvements only after you can explain why the present design exists.

Figure 3. Trace a request end to end (Lesson 1)



Ask at each hop: where is it configured? what can fail? what is logged? which test covers it?

PHP/Moodle equivalent: web server -> config.php -> require_login() -> require_capability() -> page/renderer -> $DB

6. Long-term specialization plan

C++ and quant development fit around, not instead of, the job. From week 2 onward reserve a small fixed block (for example two 60-minute sessions a week) for C++; it grows as work stabilises.

Phase	Months	C++ / DSA	Math / quant	Output
1	1-3	5.1-5.2, 20 min/day of problems	Reactivate algebra, logs, sums	CLI tool, smart-pointer lab
2	3-6	5.3-5.5; implement vector, hash map, heap; sanitizers	6.1 probability, expectation	Data-structure library + tests
3	6-12	5.6; threads, profiling; order book	6.2-6.3 linear algebra, statistics, Monte Carlo	Order book + benchmark report
4	12-24	Numerical library, market-data processor	6.4-6.5 pricing, risk, backtesting	Capstones C, D, E

Preview (C++ track): who owns this memory, and when does it die?

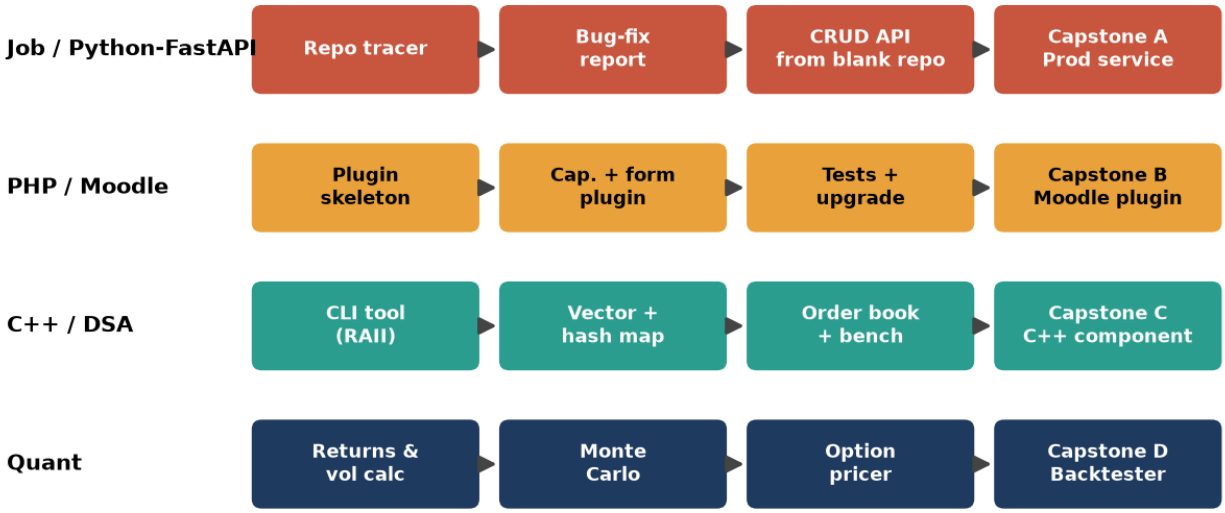


Scope exit -> ~vector() frees the block (RAII). push_back may reallocate -> old references/iterators invalid.

Quant careers differ: **research** (models, statistics), **development** (systems, performance, correctness), **trading** (decisions and risk) and **analysis** (pricing and risk support). This plan targets development, with enough mathematics to validate what you build. Mastery here takes years; the plan tracks evidence, not guarantees.

7. Project roadmap

Figure 5. Project roadmap: difficulty rises left to right



Capstone E (cross-language) unlocks after C and A. AI-dependence audits every ~6 weeks.

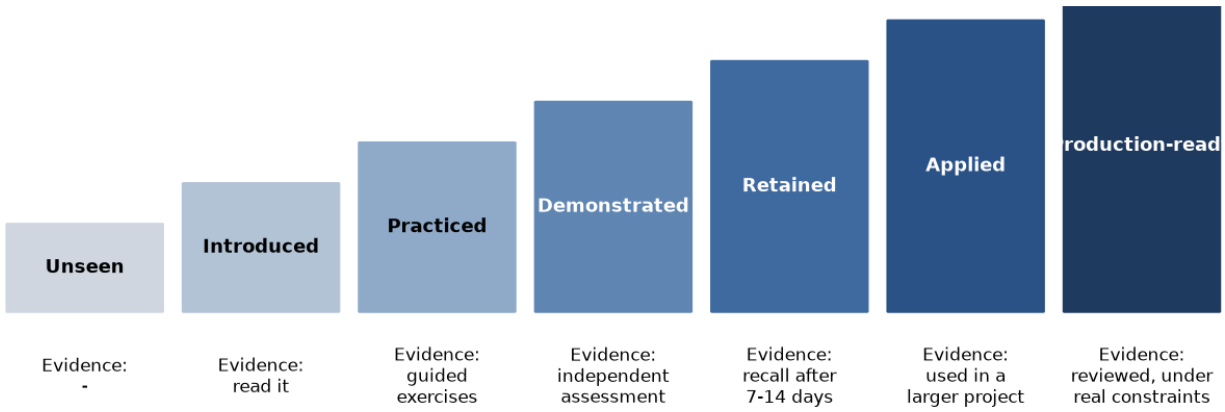
Capstone	Blank-repo build	Must show	No-AI challenge
A: Prod backend service	FastAPI + Postgres	Auth, validation, migrations, tests, logging, health checks, timeouts, docs	Rebuild one endpoint and its tests in 90 min
B: Moodle plugin	Local plugin from scratch	Capabilities, forms, persistence, privacy API, PHPUnit, upgrade step	Add a new capability-gated page
C: C++ component	CMake project	Clear interface, sanitizers clean, benchmarks, profile	Fix an injected UB bug
D: Quant research	Data -> stats -> strategy -> backtest	No look-ahead, costs, out-of-sample, stated limitations	Reproduce a published result
E: Cross-language	Python or PHP calling C++	Justification, boundary costs, deployment plan	Explain boundary failures

Required fields in every project spec

Problem statement, context, requirements, constraints, acceptance criteria, edge cases, quality requirements, deliverables, difficulty (Foundation/Intermediate/Advanced/Expert), estimated effort, progressive hints, rubric, extension challenge and reflection questions. No reference solution is shown before a serious attempt.

8. The accountability system

Figure 4. The competency ledger: status requires evidence, not self-report



100-point rubric (pass = 80 with no critical failure)

Criterion	Points
Correctness, including edge cases	25
Conceptual understanding (oral/written defense)	15
Test quality (behavior and invariants, not coverage %)	15
Independence (mode A/B vs C)	10
Readability and maintainability	10
Security and reliability (where relevant)	10
Performance with measurement (where relevant)	5
Adapting to a changed requirement	10

Critical failures (automatic fail): data loss or corruption, a security hole in the delivered feature, tests that cannot fail, or inability to explain your own code. On failure: I name the exact gap, set a focused remedial exercise, and you re-attempt.

Weekly review (10 minutes)

```
Week of: ____      Study hours (cap 10-12): ___
Lessons completed: ___   Projects delivered: ___   Independent exercises passed: ___
Work done without AI: ___   With AI and independently verified: ___
Recalled after delay (list): _____________________
Weak concepts: ___________   Repeated mistakes: ___________
Bugs investigated (-> failure log): ___
Next week's top 3 objectives: 1 ______ 2 ______ 3 ______
```

Failure log entry

```
1 Expected   2 Actual   3 Repro steps   4 Evidence (logs, output, traces)
5 Root cause   6 Why my first theory was wrong   7 Fix
8 Regression test/safeguard   9 Lesson for next time
```

AI-dependence audit (every ~6 weeks)

Pick something built with AI help. In Mode A: explain the architecture, rebuild one component, change one behavior safely, diagnose an injected bug, write tests. Anything you cannot do stays classified **unproven** and returns to the ledger at *Introduced*.

9. Lesson 0.1: Enter an unfamiliar codebase

Priority P0. Mode B. Time: 2-3 hours. Python/FastAPI and PHP/Moodle both covered.

1. Engineering problem

It is day one. The ticket says: *"Users see a 500 when the course list has more than 50 entries."* You do not know where the code lives, how to run it or what "the course list" means. Developers who dive into edits without a map change the wrong file, break unrelated behavior and cannot explain their fix. The skill is building a mental map fast and verifying it with evidence.

2. Learning objectives

- Set up a repo from its documented workflow and record each deviation.
- Locate the entry point, routing, configuration, data layer and tests for a given feature within 30 minutes.
- Trace one request from the network boundary to the database and name what each layer is responsible for.
- Write a one-page map and a list of verified/unverified assumptions.
- Reproduce a bug with a failing test before changing code.

3. First-principles explanation

Any web application is a pipeline: **a request enters, is routed, validated, authorized, handled, persisted and answered**. Frameworks differ in naming but not in structure. Your map has five questions:

Question	FastAPI places to look	Moodle places to look
Where does execution start?	main.py / app factory; `uvicorn app.main:app`; Dockerfile CMD	A page script (e.g. `/local/x/index.php`) or web service endpoint; `config.php`
How is a URL mapped?	`@router.get(...)`, `include_router`	File path = URL; `db/services.php` for web services
Who can do this?	`Depends(get_current_user)`, scopes/roles	`require_login()`, `require_capability()`, `db/access.php`
Where is data read/written?	ORM models, repository/CRUD module, raw SQL	`$DB`, persistent classes, `db/install.xml`
How is config injected?	.env, settings class, env vars, secrets store	`$CFG`, admin settings, plugin config

```
# Discovery commands (read-only, safe)
git log --oneline -n 20 && git ls-files | head -100
grep -rn "include_router\|@router\." src/            # Python routing
grep -rn "require_capability\|require_login" local/   # Moodle access checks
grep -rn "DATABASE_URL\|getenv\|os.environ" .        # config discovery
```

4. Worked example (trace on paper)

Take the route below. Predict first, then check your prediction against the code.

```
@router.get("/courses", response_model=list[CourseOut])
def list_courses(limit: int = 50, db: Session = Depends(get_db),
                 user: User = Depends(current_user)):
    return db.query(Course).filter(Course.owner_id == user.id).limit(limit).all()
```

- Request `GET /courses?limit=500` is parsed: query param coerced to int (422 if not).
- `get_db` and `current_user` run **before** the handler. A failure there never reaches it.
- ORM builds SQL with `WHERE owner_id = ? LIMIT ?`; rows become `Course` objects.
- `response_model` validates/serializes output. A bad row (e.g. a null field the model forbids) raises a 500 *after* the handler ran.

This last point is the kind of hypothesis you test: could the 500 at >50 entries be a serialization failure on row 51, not a limit problem at all? You do not know yet, and that is why you reproduce first.

5. Production perspective

- **Correctness:** fix the root cause, not the symptom. **Security:** the trace shows exactly where authorization is (or is not) checked.
- **Operations:** note where logs go and which request id ties logs together.
- **Convention:** match naming, error format and test layout already present, even if you would do it differently.

6. Failure laboratory

Predict before reading on. In the route above the developer replaces `Depends(get_db)` with a module-level `db = SessionLocal()`. All unit tests still pass. Which production failures appear and why do the tests not catch them? *(Think: concurrency, session lifetime, transaction state after an exception, test isolation.)*

Moodle version: a page calls `require_login()` but forgets `require_capability()`. Which user can reach it, and what test would prove it?

7. Retrieval practice (close the document)

- List the five discovery questions from memory.
- Which runs first: the handler or its dependencies? Why does it matter for error handling?
- Where in Moodle is a capability declared, and where is it checked?
- Name three places configuration could come from, and the command that finds each.
- Why must you reproduce a bug before changing code?

8. Independent implementation challenge (Mode A, 45 min)

Write a script `repo_map.py` that takes a repo path and prints: top-level layout, detected framework (FastAPI/Django/Moodle), all route decorators with file:line, and every environment-variable lookup. Pure standard library. Handle missing directories, binary files and huge files without crashing. Write 3 tests.

9. Assignment: the repository dossier (Mode B, Foundation, 2-3 h)

Field	Specification
Problem	Produce a reusable day-one dossier for an unfamiliar codebase.
Context	You will repeat this on Monday; the template is your tool.
Requirements	Choose a public FastAPI repo and a Moodle core plugin. For each: setup notes, run log, one traced request/page, test run, config list, 5 open questions.
Constraints	No AI. Read-only on the repo. Time-box: 90 minutes per repo.
Acceptance	A reader could set up and find the code for any feature using only your dossier.
Edge cases	Missing README, failing tests on a clean checkout, secrets required, multiple apps in one repo.
Quality	Every claim marked *verified* (with command or file:line) or *assumed*.
Deliverables	dossier.md x2, one diagram per repo, repo_map.py with tests.
Effort / level	2-3 h / Foundation.
Hints (progressive)	1) Start from the CI config: it shows what really runs. 2) Search for the router registration. 3) Follow imports downward.
Extension	Run the app with a debugger and breakpoint inside a dependency; screenshot the call stack.
Reflection	Which assumption was wrong? Which discovery command saved the most time?

10. Engineering review checklist

- Did I run it, or only read it? Did I record each setup deviation?
- Can someone reproduce my trace?
- Are assumptions labelled? Did I touch anything I should not have?

- Do my tests cover missing path, binary file, huge file?

11. Mastery gate (pass = 80/100)

Criterion	Pts
Dossier accuracy (verified claims)	30
Trace correctness and completeness	20
repo_map.py correctness + 3 meaningful tests	20
Oral defense: explain any step without notes	20
Assumption discipline, no unexplained changes	10

12. Reflection (5 lines)

What I learned. What I misunderstood. What took the longest. Bugs and how I diagnosed them. What remains unclear. Enter any failure in the failure log and set status in the ledger: *Introduced* now; *Demonstrated* only after the gate.

References

- FastAPI docs: fastapi.tiangolo.com (Tutorial: Dependencies; Bigger Applications).
- Moodle developer docs: moodledev.io (Plugin files, Access API, Database API).
- Python docs: docs.python.org/3 (Tutorial; Data model).

Next: send diagnostic answers and your dossier. I will grade them, set the ledger and adjust Lessons 0.2-1.4.